

Model Intelligence Part 1

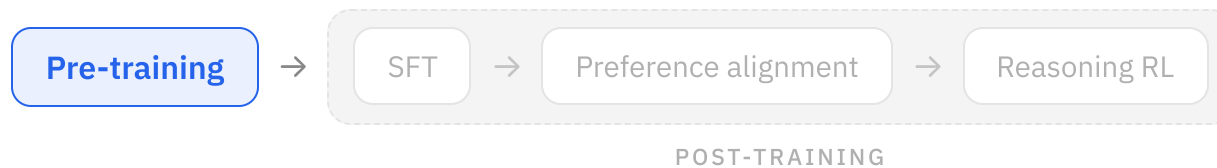
Pre-Training & Post-Training

AI Tech Review | Deena | June 26, 2026

PART 1

Pre-Training

How raw text becomes a **base model**



PRE-TRAINING

What is pre-training?

Goal: next-token prediction.

- ▶ Most modern LLMs are based on the transformer architecture.
- ▶ Pre-training involves feeding the transformer enormous amounts of raw text, and learning to predict the next token.
- ▶ **Self-supervised:** the "label"/"target" for each position is just the token that actually comes next in the sequence.
- ▶ The result is a **base model** that excels at predicting the next token in a sequence by "pattern matching" with its training data.

Everything in pre-training reduces to one question: *given the tokens so far, what comes next?*

Tokenization

Tokens are the atomic units fed to a transformer. Text is split into tokens, and each token maps to an integer ID.

The vocabulary of an LLM is the set of all tokens it can take as input and predict. Typical vocabulary sizes $d_{\text{vocab}} \approx 50\text{K}$ (GPT-2) to 100K+ (modern models).

Special tokens are used to represent the start of a sequence, the end of a sequence, and padding.

Tokenized text →

Token	ization	is	just	chunk	ing	text	.
-------	---------	----	------	-------	-----	------	---

Integer IDs → [30642, 1634, 318, 655, 16058, 278, 2420, 13]

Common tokenization strategies include:

- ▶ **Byte-pair encoding (BPE):** break text into subwords, then merge the most frequent pairs to create a vocabulary.
- ▶ **Word-level tokenization:** split text into whole words.
- ▶ **Character-level tokenization:** split text into individual characters.

After tokenization, a piece of text becomes a sequence $x_1, x_2, \dots, x_t$, where each x_i is an integer $\in [0, d_{\text{vocab}} - 1]$.

Next-token prediction: a distribution over the vocabulary

Given the tokens so far, the model predicts what comes next.

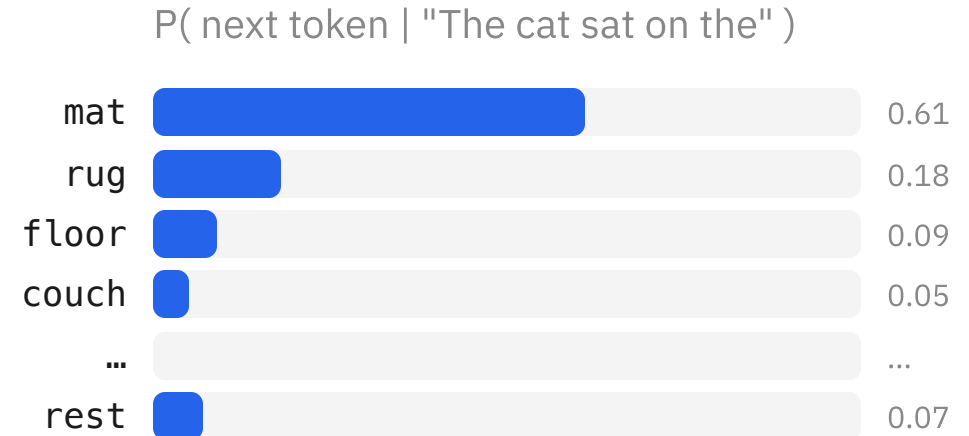
The cat sat on the → **mat**
 x_1 x_2 x_3 x_4 x_5 $x_6?$ most likely

The output of the model is a probability distribution over all possible tokens in its vocabulary.

Formally, the model maps the input $x_1, \dots, x_t$ to a d_{vocab} -dimensional vector in which coordinate i is the probability that token i comes next given the tokens that came before it — the raw scores that come out of the model's final layer (**logits** $z \in \mathbb{R}^{d_{\text{vocab}}}$) passed through a **softmax**:

$$P(x_{t+1} \mid x_1, \dots, x_t) = \text{softmax}(z)$$

Inference = sample one token from this vector, append it, repeat. That loop is **autoregression**; temperature / top-p tune how adventurous it is.



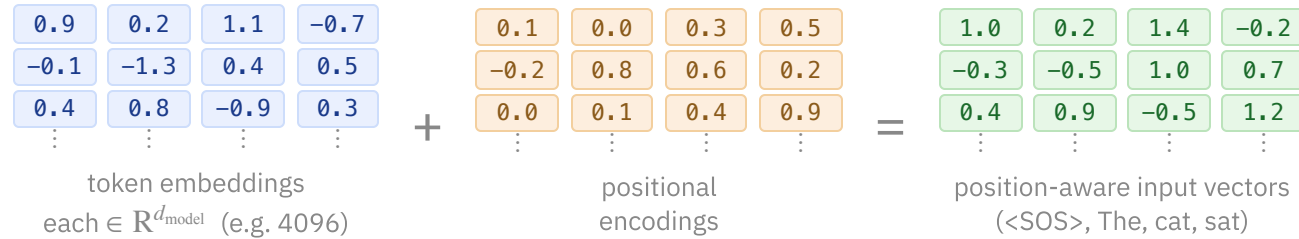
Inside the transformer — 1: from text to vectors

The cat sat

↓ **tokenizer** — prepend a special <SOS> start token, split the rest into tokens, map each to an integer ID

<SOS> 464 3797 3332

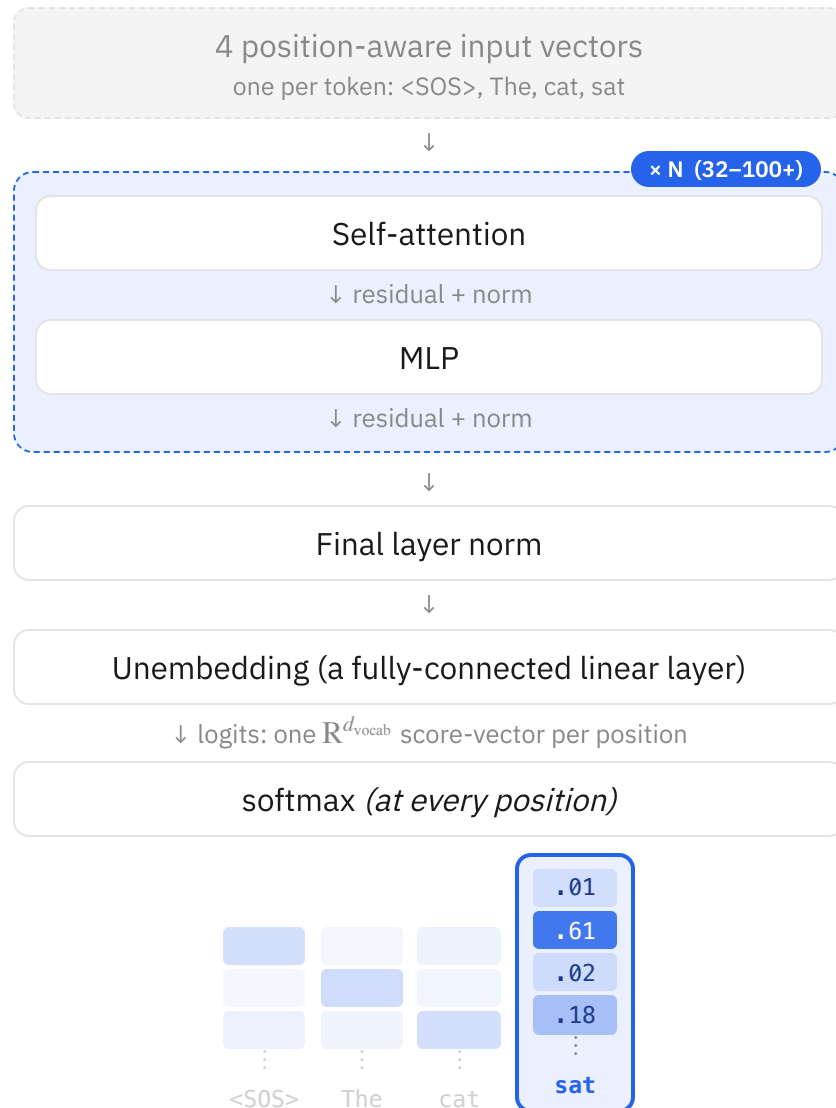
↓ **embedding lookup**, then + **positional encoding** (so the model knows token order)



↓ these vectors enter the transformer stack...

Embedding = a learned vector per token (the model's "vocabulary" of meanings). One vector per token flows in — and one per token flows out of every layer that follows. The <SOS> token gives the model a slot to predict the very first real token from.

Inside the transformer — 2: the transformer stack



Self-attention

Each position pulls in information from other positions, via query·key dot products (KQV). This is how context flows between tokens.

MLP

An independent per-position transform — where most factual knowledge is thought to live.

Residuals + layer norm

Neural network architectural tricks that stabilize gradients and help gradients flow through deep layers!

[arXiv:1706.03762](#) Attention Is All You Need — Vaswani et al. (the transformer architecture)

[jalammar.github.io](#) The Illustrated Transformer

[YouTube](#) How might LLMs store facts · Deep Learning Chapter 7

The stack emits a **full $\mathbb{R}^{d_{\text{vocab}}}$ distribution at every position**, in a single pass — each predicts the token that follows its own prefix. **Training** uses all of them (one next-token loss per position); **inference** reads only the last.

Training using cross-entropy loss

We want the model to put **high probability on the token that actually came next**. So we minimize the negative log-likelihood, averaged over every position (or equivalently, maximize the log-likelihood):

$$L(\theta) = - \sum_t \log P_{\theta}(x_{t+1} \mid x_1, \dots, x_t)$$

- ▶ θ is the model's weights, which are updated during training.
- ▶ P_{θ} is the model's output distribution
- ▶ $P_{\theta}(x_{t+1} \mid x_1, \dots, x_t)$ is the probability the model assigns to the true next token x_{t+1} given the input sequence $x_1, \dots, x_t$

Intuition: each term is $-\log(\text{probability the model assigned to the correct next-token})$.

- ▶ If the model assigns a lot of probability to the true next token, $P_{\theta}(x_{t+1} \mid x_1, \dots, x_t) \approx 1$, and $-\log(1) = 0$: no penalty.
- ▶ If the model assigns a low probability to the true next token, then the loss at timestep t is very high.
- ▶ The goal of training is to drive down the model's **"surprise"** at the real next token.

We know the whole sequence up front, so the **targets are just the input shifted left by one**:

input	The	cat	sat	on	the
target	cat	sat	on	the	mat

During training, one LLM forward pass can score all 5 predictions in parallel. Inside the attention layers, a causal mask stops each position from "peeking" ahead to the actual tokens that come after it.

PRE-TRAINING

The result: matmul-powered autocomplete

After pre-training, the base model is a brilliant next-token predictor — but it is **not** an assistant. It just continues text in the most plausible way.

Prompt

What is the capital of France?

What we actually want

The capital of France is Paris.

A base model might just continue...

What is the capital of Germany?
What is the capital of Italy? ...

(it saw lists of quiz questions in training)

- ▶ The model absorbed enormous **knowledge**, grammar, style, even some reasoning — purely as a side effect of learning to represent and predict text effectively.
- ▶ But it has no notion of being *asked* and *answering*. Teaching that behavior is **post-training**.

PART 2

Post-Training

From a base model to an **assistant**



POST-TRAINING

POST-TRAINING

From base model to assistant

Recall: a base model is **matmul autocomplete**. It absorbed grammar, style, and knowledge, but it has no notion of being *asked* a question or playing the role of an *assistant*.

A base model continues the text...

What is the capital of France?
What is the capital of Germany?
What is the capital of Italy? ...

An assistant responds.

The capital of France is Paris.

Post-training is the bridge. It teaches the base model **what role to play** when given an input — and shapes *how* it responds along the way.

POST-TRAINING

Why do we post-train?

Some common purposes of post-training are:

Behavior-shaping

"act like an assistant"

- ▶ Respond in user / assistant turns instead of continuing the prompt as raw text
- ▶ Produce valid JSON when a system prompt asks for it
- ▶ Follow length / formatting instructions
- ▶ Keep a consistent persona

Alignment

"act like a good assistant"

- ▶ When to refuse vs. comply
- ▶ Calibrated honesty — admit uncertainty rather than confabulate
- ▶ Avoid sycophancy

Capabilities

"act like a helpful assistant"

- ▶ Chain-of-thought reasoning
- ▶ Tool / function calling
- ▶ Multi-step code generation
- ▶ Long-context retrieval, agentic loops

A single post-training technique can sometimes cover multiple purposes at the same time. For example, supervised fine-tuning (SFT) can be used to train the model to follow instruction patterns, such as those asking it to generate code.

The 3 axes of post-training techniques

These are orthogonal dimensions for describing post-training techniques (i.e. the "how" of post-training). Any post-training technique can be decomposed into a combination of **Stage** × **Signal** × **Algorithm**.

Stage which step of the modern LLM pipeline this technique fits into

Base model

→

SFT

→

Preference alignment

→

Reasoning RL

Post-training techniques operate at one of the three post-training stages (SFT, preference alignment, or reasoning RL). The base model is the *input* to post-training, not a stage itself.

Signal what tells the model whether each output is good

curated examples

human preference pairs

reward-model scores

verifiable rewards

AI-generated feedback

The **signal** is the feedback the model learns from — what each output gets compared to so we can compute a weight update. It can take three forms:

- a *label* — the exact right answer the model should have produced (used in SFT)
- a *preference* — a comparison like "output A is better than output B" (used in RLHF / DPO)
- a *scalar reward* — a single number scoring how good an output was, e.g. 0.87 from a learned reward model, or $1.0 / 0.0$ from a unit-test verifier (used in RL methods)

Algorithm how the signal is turned into a weight update

cross-entropy SFT

PPO

DPO

GRPO

REINFORCE

rejection sampling

"**Optimizing**" means iteratively adjusting the model's weights θ so that the model's outputs score higher / result in lower loss. Algorithms differ in *how* they compute that update — e.g. SFT minimizes a supervised loss over labels; PPO / GRPO / REINFORCE are RL methods that sample outputs and push the model toward higher-reward ones; DPO derives a loss directly from preferences (no RL loop); rejection sampling filters good outputs and trains on them like SFT.

POST-TRAINING

A caveat on the "stage" dimension: not every model goes through the full pipeline



Most models skip stages. A few example paths:

Older chat models (early Llama-Instruct, early Claude / GPT)



Reasoning models (o1, R1, DeepSeek-V3.1)



R1-Zero (more on R1-Zero later!)



Domain-specialized (e.g. a coding-only fine-tune)

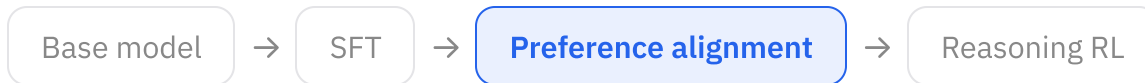


POST-TRAINING

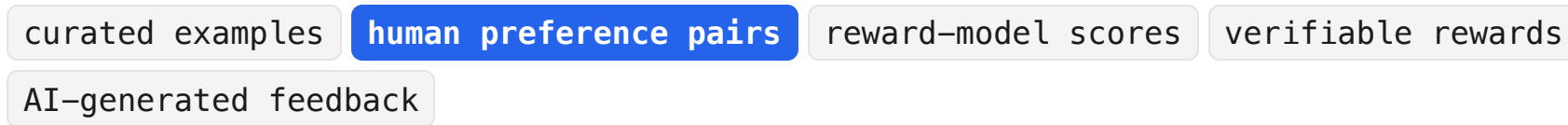
Worked example: where does RLHF land on the 3 axes?

RLHF (Reinforcement Learning from Human Feedback) is just one recipe drawn from these axes:

Stage



Signal



↳ human-labeled preference pairs, transformed into scalar rewards by a learned **reward model** based on the Bradley-Terry probabilistic framework.

Algorithm



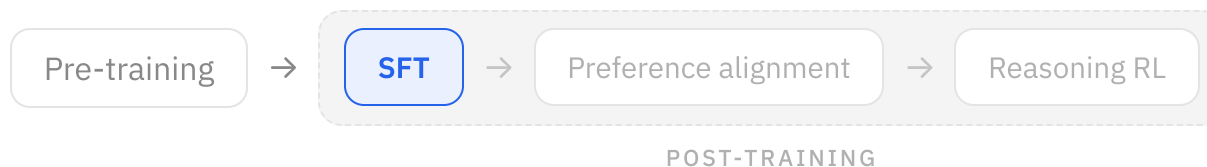
RLAIF = the same recipe, but preference labels come from an AI judge instead of humans.

RLVR = same shape, but the signals come from a *verifier* (unit tests, exact-match) instead of a learned reward model.

PART 3

Supervised Fine-Tuning

Teaching the base model to **imitate** curated responses



Supervised Fine-Tuning (SFT)

Just like in pre-training, SFT also uses a cross-entropy loss and trains the model on targets which consist of the input sequence shifted over by one token. However, two things change:

- ▶ **Data:** structured (prompt, response) pairs formatted with chat-template tokens (e.g. `<|user|>` and `<|assistant|>`), instead of raw text.
- ▶ **Mask:** the loss is summed *only over the response tokens*. The system and user prompt are just considered inputs, not outputs we want the model to learn to predict. They are masked so that any differences in the model's probability distribution on the prompt tokens don't contribute to the loss function.

What an SFT training example looks like:

```
<|user|>What is the capital of France?<|end|>
<|assistant|> The capital of France is Paris.
<|end|>
```

■ prompt tokens — loss masked
 ■ response tokens — loss applied

The SFT loss:

$$L_{\text{SFT}}(\theta) = - \sum_{t \in \text{response}} \log P_{\theta}(x_{t+1} \mid x_1, \dots, x_t)$$

Identical to the pre-training loss from slide 8 — the sum just runs over response positions only.

SFT datasets are usually much smaller than pre-training datasets — **10K–1M examples** vs. trillions of tokens. The base model already knows language; SFT just teaches it **what role to play** and **which continuations are appropriate** by showing it the right chat-templated examples.

SFT — on human, distilled, or self-generated data — only teaches the model to *imitate*. Preference alignment and RL can teach

models what's good vs. bad, and push them **beyond** their training distribution.

PART 3 · SFT

Different SFT datasets for different purposes

Instruction tuning

"act like an assistant"

Chat datasets, Q&A pairs, dialogue. Teaches the user / assistant turn structure, helpful tone, basic refusals, and formatting conventions.

Examples: Alpaca, FLAN, ShareGPT, OpenAssistant (OASST), HH-RLHF SFT split.

→ behavior-shaping

Capability-specific SFT

"act like a useful assistant"

Worked-example traces of specific skills the base model can't pattern-match on its own. The student inherits the skill by imitating those traces.

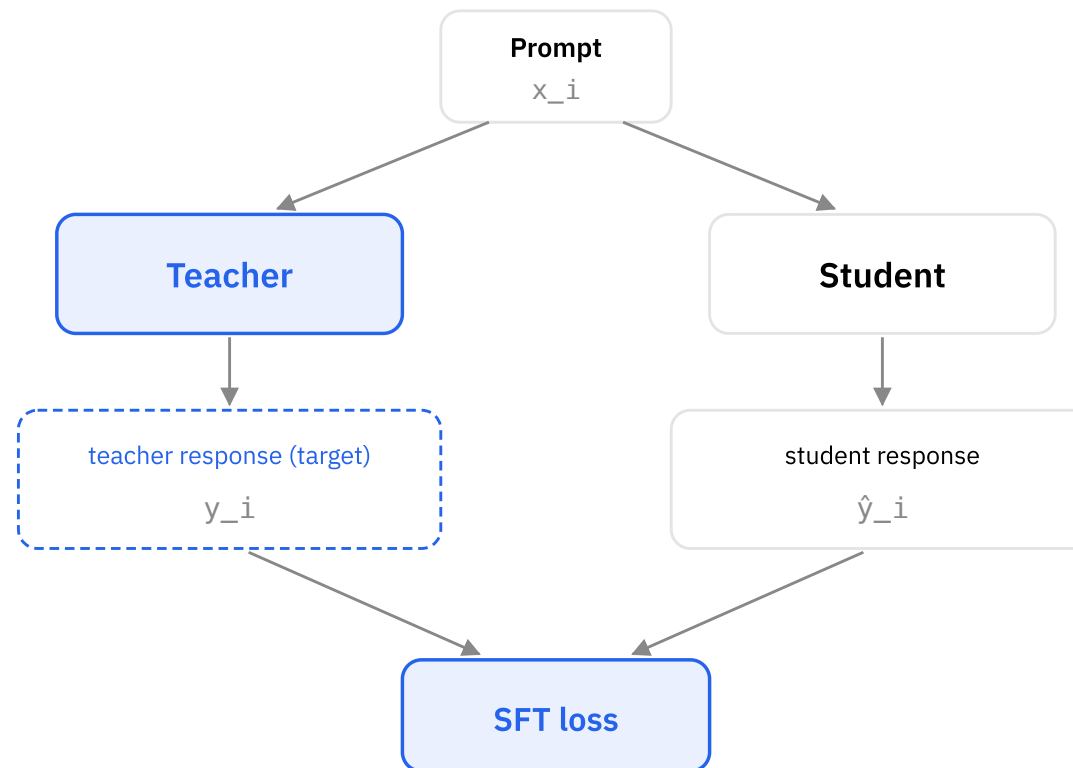
Examples: function-calling traces, code-execution traces, long-context retrieval examples, chain-of-thought reasoning traces.

→ capabilities

These aren't either / or. Modern SFT recipes mix instruction tuning and several capability slices in one run — sometimes hundreds of thousands of mixed examples, weighted by domain.

Distillation as a form of SFT

A model (called the **student**) is trained on data produced by another model (called the **teacher**, usually a stronger or larger one). The student learns to mimic the teacher.



For each prompt x_i , the teacher's response y_i is the SFT target — the student is trained to produce $\hat{y}_i$ matching y_i .

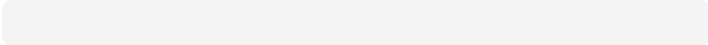
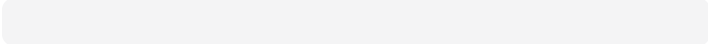
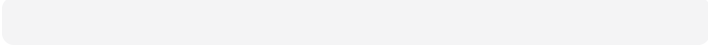
Distillation follows the same principles as SFT, but in distillation, the targets (what we want the student model to learn to output) come from another model.

Sequence-level (data) distillation

The teacher and student each generate a response to the same prompt. At every position in the output, we use the teacher's actual outputted token as the target, with **standard cross-entropy loss** as the loss function. The target is one-hot encoded, meaning we assign a probability of 1 to the teacher's outputted token and 0 to all other tokens.

When we calculate the loss for the student trying to predict the next token after the prefix "...the capital of France is", the target is a **one-hot** on the teacher's "chosen" token:

The teacher's probability density is concentrated on a single token

Paris		1.00
Lyon		0.00
the		0.00
...		0.00

Student's current distribution

Paris		0.32
Lyon		0.22
the		0.18
...		0.28

Cross-entropy loss at this position:

$$L_{\text{pos}} = -\log P_{\text{student}}(\text{Paris} \mid \text{prefix}) = -\log(0.32) \approx 1.14$$

Summed across all response positions: $L_{\text{seq}} = -\sum_t \log P_{\text{student}}(y_t \mid y_{<t}, x)$, where y_t is the teacher's sampled token at position t .

Across training iterations: the target one-hot's are *fixed* — the teacher generates the response once, and it's reused for every iteration during SFT. The student's distribution gradually updates to put all its mass on Paris.

Logit-level (classical) distillation

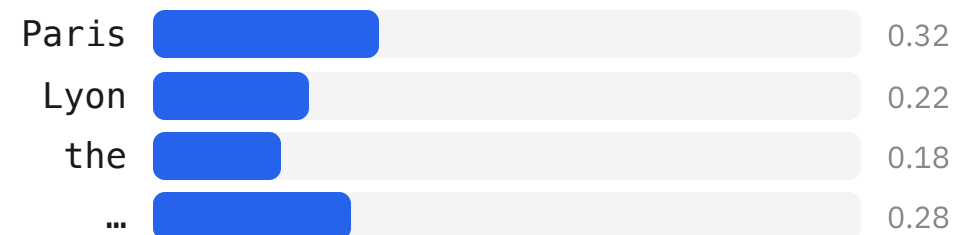
Assumes we have access to the teacher's **full probability distribution** (the probability assigned to every possible tokens) at every position in the response. The student learns to match that distribution, typically via **KL divergence**.

At the same position predicting after "...the capital of France is", the target is now a **soft distribution**:

Target — teacher's full distribution



Student's current distribution



KL divergence between teacher and student probability distributions at position t :

$$L_t = \text{KL}(P_{\text{teacher}}(\cdot \mid x_1, \dots, x_t) \parallel P_{\text{student}}(\cdot \mid x_1, \dots, x_t)) = \sum_{w \in V} P_{\text{teacher}}(w \mid x_1, \dots, x_t) \log \frac{P_{\text{teacher}}(w \mid x_1, \dots, x_t)}{P_{\text{student}}(w \mid x_1, \dots, x_t)}$$

The " $\cdot$ " in $P(\cdot \mid x_1, \dots, x_t)$ is shorthand for the **full distribution over the vocabulary** (i.e. the probability of each token in the vocabulary) at position t . The expanded form sums over every token $w \in V$. Both sides are full probability vectors. Total loss summed across positions: $L_{\text{logit}} = \sum_t L_t$.

Across training iterations: the teacher's distribution is *fixed*. The student's probability distribution gradually updates to match the entire *shape* of the teacher's, inheriting the teacher's uncertainty and latent knowledge about plausible alternatives.

Reasoning trace distillation

Reasoning trace distillation is sequence-level distillation with a twist: A strong **teacher** reasoning model solves hard problems and writes long chain-of-thought traces — the `<think>` reasoning plus the final answer. Those full traces become **fixed SFT targets**. A smaller **student** model is trained with next-token prediction on every token in the trace and answer (the loss below).

One training example (a math problem):

```
<|user|>What is 23 × 47?<|end|>  
<|assistant|> <think>  
23 × 47 = 23 × (50 - 3) = 1150 - 69 = 1081  
</think>  
The answer is 1081.<|end|>
```

↑ the entire chain-of-thought *and* the final answer are loss-applied response tokens.

Loss summed across all trace tokens (a trace may be hundreds of tokens long):

$$L_{\text{trace}} = - \sum_{t \in \text{trace} + \text{answer}} \log P_{\text{student}}(y_t \mid y_{<t}, x)$$

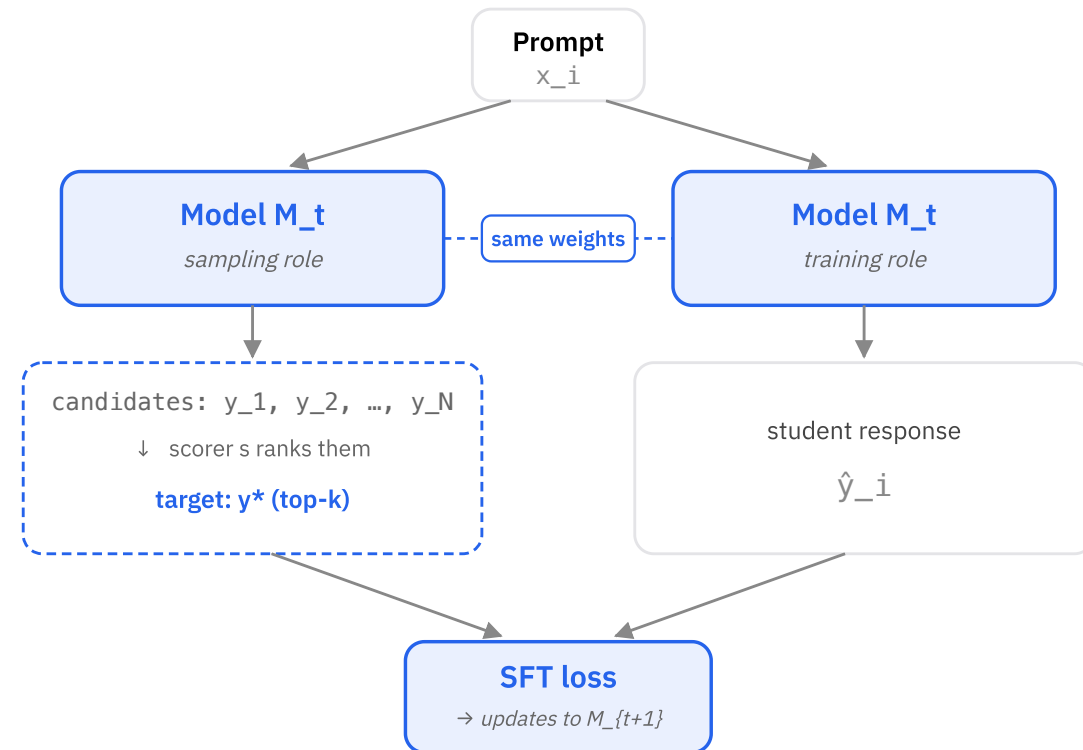
Across iterations: the full trace is *fixed*. Because the loss covers every token, the student gradually learns to reproduce the *reasoning style itself*, not just the final answer.

Self-distillation

Self-distillation is distillation where the model acts as its own teacher — at each round, the teacher and student are the same weights, and supervision comes from the model's own rollouts:

- ▶ **Sample** — the current model M_{θ_t} generates one or more candidate responses (rollouts) per prompt.
- ▶ **Select or construct** — turn those rollouts into *good supervision targets*: **select** survivors via a scorer (verifier, judge, rubric pass rate, top- k), or **construct** targets from the same model with privileged context (reference solution, rubric, self-reflection).
- ▶ **Train** — update the student to match those targets: SFT on chosen sequences, or token-level distillation from the model's privileged view $\rightarrow M_{\theta_{t+1}}$.
- ▶ **Repeat** — $M_{\theta_{t+1}}$ becomes the new M_{θ_t} ; resample and repeat. Unlike other distillation flavors, targets refresh every round.

The same loop (sample, select/construct, train, repeat) can be used for **rejection-sampling SFT**, **rubric-guided** training, and **reflective** on-policy distillation (ROSD) — only the select/construct step changes.



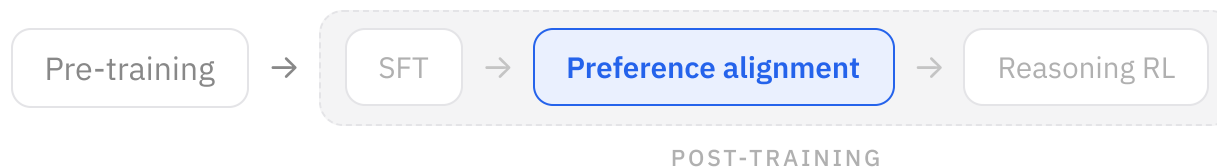
M_{t+1} becomes the new M_t in the next round; the cycle repeats with refreshed targets.

Across iterations: this is the only distillation flavor where *the targets themselves change every round*. As the model improves, its rollouts improve, and the selected or constructed supervision gets better too — a feedback loop that makes self-distillation **RL-adjacent** even when each step is imitation or distillation, not policy gradients.

PART 4

Preference-based alignment

Teaching models what's **good** using preferences over a set of outputs



The setup: pairwise preferences

Recall from Part 3: SFT teaches the model to imitate curated targets. But for questions like "is this response *helpful?* *honest?* *safe?* *well-formatted?*" there's no single objective correct sequence of tokens to imitate.

What we *can* get is **comparative judgments**: given two candidate responses to the same prompt, which one does a human (or AI) prefer?

Prompt (x): How should I prepare for a job interview?

Response A (y_w): Research the company, prepare specific examples from past experience, practice common questions out loud, and prepare thoughtful questions for the interviewer...

Response B (y_l): Just be yourself. You'll do fine!

Preference: $A > B$ (Response A is preferred over Response B)

The canonical preference datapoint is a triple (x, y_w, y_l) : prompt, **chosen** ("winner"), **rejected** ("loser"). Some methods/datasets use full rankings or k-wise comparisons, but everything reduces to pairs for the math below.

Reward modeling — Bradley-Terry

Most RL methods need a **scalar reward** per (prompt, response) to represent how good a response to the given prompt is. But our data is a triple of (prompt, winner, loser), not scalars. We need a way to convert preferences into a learnable reward signal.

Solution: train a **reward model** $r_\phi(x, y) \rightarrow \mathbf{R}$ that scores responses s.t. preferred responses get higher scores.

Bradley-Terry is a probabilistic framework for pairwise comparisons. It assumes that every item has a latent score s_i , and that the probability that item i beats item j can be explained as the sigmoid of their score difference: $P(i > j) = \sigma(s_i - s_j)$.

Reward modeling tries to reverse engineer these latent scores by learning a function $r_\phi(x, y) \rightarrow \mathbf{R}$ that scores responses such that we recover the Bradley-Terry assumption:

$$P(y_w > y_l \mid x) = \sigma(r_\phi(x, y_w) - r_\phi(x, y_l))$$

Bigger reward gap $\rightarrow$ higher probability that y_w is preferred over y_l

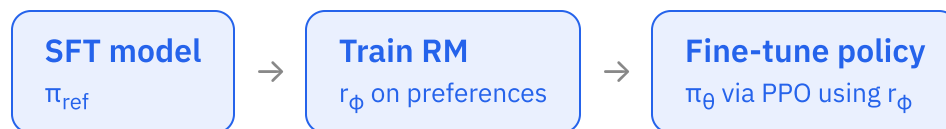
RM training loss — negative log-likelihood using the Bradley-Terry assumption, over the preference dataset $\mathbf{D}$:

$$L_{\text{RM}}(\phi) = -\mathbb{E}_{(x, y_w, y_l) \sim \mathbf{D}} [\log \sigma(r_\phi(x, y_w) - r_\phi(x, y_l))]$$

The reward model is a transformer — typically initialized from the SFT model with a **scalar head** (a linear layer replacing the LM head). The RM takes in a (prompt, response) pair and outputs a single number. When training the RM, we give it a preference triplet (x, y_w, y_l) , make two separate forward passes: $r_\phi(x, y_w)$ and $r_\phi(x, y_l)$, and compute the training loss for the RM based on their gap. It is a *learned* signal, not ground truth.

Classical RLHF — putting it together

InstructGPT's RLHF recipe consists of: collecting human preference pairs → training a reward model → fine-tuning the policy using the RM as the reward signal (canonically PPO, although other weight-update algorithms are also used).



LLM ↔ RL mapping: the **policy** is the model being trained; the **state** s_t is the prompt + tokens so far; the **action** a_t is the next token; a full response is a **trajectory**; the **reward** $r_\phi(x, y)$ is assigned once per trajectory.

The **RLHF objective** being maximized over the policy π_θ :

$$\max_{\pi_\theta} \mathbb{E}_{x \sim D, y \sim \pi_\theta(\cdot | x)} [r_\phi(x, y) - \beta \text{KL}(\pi_\theta(\cdot | x) \parallel \pi_{\text{ref}}(\cdot | x))]$$

Expected reward (under the current policy) minus a **KL penalty** against the SFT-ed reference policy π_{ref} . Without the KL, the policy can drift far from a sensible language model and collapse to high-reward gibberish.

PPO is the algorithm that updates π_θ to maximize the objective above:

$$\mathcal{L}^{\text{CLIP}}(\theta) = \mathbb{E}_t [\min(\rho_t(\theta) A_t, \text{clip}(\rho_t(\theta), 1 - \epsilon, 1 + \epsilon) A_t)]$$

$\rho_t(\theta) = \pi_\theta(a_t | s_t) / \pi_{\theta_{\text{old}}}(a_t | s_t)$ is the **policy ratio** (how much more or less the new policy favors that token vs. the previous iteration); A_t is the **advantage** (how much better the action did than expected, derived from r_ϕ minus a learned baseline). The clip $[1 - \epsilon, 1 + \epsilon]$ stops the ratio from blowing up in one update — bounding per-step change is what keeps PPO stable on the noisy gradients you get from RL rollouts.

DPO — skip the reward model

The KL-regularized reward-maximization objective has a closed-form solution for the optimal policy π^* that can be written in terms of π_{ref} and r . With some math, we can derive a loss function that only depends on the learnable policy π_θ , with π_{ref} as a fixed reference. No RM needed to train!

$Z(x) = \sum_y \pi_{\text{ref}}(y \mid x) \exp(r(x, y)/\beta)$ is the **partition function** — it normalizes π^* into a valid probability distribution over all possible responses y . Generally intractable to compute (a sum over the response space), but conveniently it cancels in step (3), so we never need to.

$$(1) \text{ Optimal policy: } \pi^*(y \mid x) = \frac{1}{Z(x)} \pi_{\text{ref}}(y \mid x) \exp\left(\frac{1}{\beta} r(x, y)\right)$$

$$(2) \text{ Solve for } r: \quad r(x, y) = \beta \log \frac{\pi^*(y \mid x)}{\pi_{\text{ref}}(y \mid x)} + \beta \log Z(x)$$

$$(3) \text{ Sub into BT: } P(y_w > y_l \mid x) = \sigma\left(\beta \log \frac{\pi^*(y_w \mid x)}{\pi_{\text{ref}}(y_w \mid x)} - \beta \log \frac{\pi^*(y_l \mid x)}{\pi_{\text{ref}}(y_l \mid x)}\right)$$

Maximum log likelihood on $\mathbf{D}$ (parameterizing π^* as the trainable π_θ) gives the **DPO loss**:

$$L_{\text{DPO}}(\theta) = -\mathbb{E}_{(x, y_w, y_l) \sim \mathbf{D}} [\log \sigma(\beta \log \frac{\pi_\theta(y_w \mid x)}{\pi_{\text{ref}}(y_w \mid x)} - \beta \log \frac{\pi_\theta(y_l \mid x)}{\pi_{\text{ref}}(y_l \mid x)})]$$

Why DPO was a big deal: Skips the separate RM training run and PPO's iterative rollout-and-update cycle. Instead, DPO is just SGD on a fixed preference dataset $\mathbf{D}$.

The DPO family

DPO inspired a wave of variants that fix specific issues or relax specific assumptions.

KTO

Kahneman-Tversky Optimization

No preference *pairs* needed — just **binary labels** ("good" / "bad") per individual response.

When to use: production thumbs-up/down data, no paired comparisons. Common in real deployments.

[arXiv:2402.01306](#)

KTO: Model Alignment as Prospect Theoretic Optimization — Ethayarajh et al.

ORPO

Odds-Ratio Preference Optimization

Merges SFT and preference optimization into a **single training stage** with a loss that combines L_{SFT} and $L_{\text{preference}}$ (pushes model towards winning response and away from losing response).

When to use: faster recipe; skips the separate SFT checkpoint requirement.

[arXiv:2403.07691](#)

ORPO: Monolithic Preference Optimization without Reference Model — Hong et al.

IPO

Identity Preference Optimization

Fixes a known DPO failure mode: the chosen-vs-rejected gap can grow unbounded, **over-fitting** to specific pairs.

Fix: adds a regularizer that anchors the gap.

[arXiv:2310.12036](#)

A General Theoretical Paradigm to Understand Learning from Human Preferences — Azar et al. (IPO)

All three are still "preference alignment" — they differ in *what data they need* (KTO), *how they integrate with SFT* (ORPO), or *how they regularize* (IPO). The DPO loss is the common ancestor.

RLAIF and Constitutional AI

Up to now we've assumed humans label preferences. But human labeling is **slow, expensive, and noisy** across labelers. An alternative is to swap in an AI as the labeler.

RLAIF

RL from AI Feedback

Same recipe as RLHF (or DPO) — but the preference labels are produced by an **AI judge** instead of humans. Scales much further; consistency is higher (one model's biases instead of many labelers' noise).

Trade-off: the AI judge's blind spots become *your* blind spots.

arXiv:2309.00267

RLAIF: Scaling RL from Human Feedback with AI Feedback — Lee et al.

Constitutional AI

a specific RLAIF recipe (Anthropic, 2022)

The AI judge isn't free-form — it scores responses against a **written constitution**: principles like "be helpful," "avoid harmful instructions," "admit uncertainty." Two-phase recipe:

- ▶ **Supervised learning phase:** the model critiques and revises its own outputs against the constitution.
- ▶ **RL phase:** AI preferences derived from constitution-conditioned judgments train the policy.

Why structured: the constitution is auditable and editable — a transparent alternative to opaque labeler disagreement.

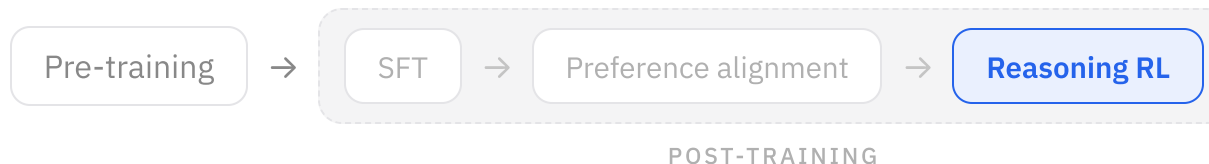
arXiv:2212.08073 Constitutional AI: Harmlessness from AI Feedback

RLAIF is the technique family; Constitutional AI is one specific recipe within it. Both are now standard in scalable preference-alignment pipelines.

PART 5

Reinforcement Learning from Verifiable Rewards

Same RL shape as RLHF — but the signal is a **verifier**, not a learned reward model



Why drop the reward model?

Reward models work, but they're a **learned approximation** of human preference — and learned approximations have systematic failure modes that get *worse* the more you optimize against them.

Reward hacking: the policy finds responses that score high on the RM without actually being good.

Distribution shift: the RM is trained on a snapshot of model outputs, but the policy drifts during optimization. The RM is increasingly grading out-of-distribution responses, and may need to be retrained on how to assign scores to a new spectrum of responses.

Noisy labels & inherited biases: human preference data is inconsistent labeler-to-labeler, and the RM also learns systematic human biases. E.g. *length bias*: annotators slightly prefer longer responses, so RLHF-trained models tend to grow longer over training.

Quality capped by human judgment: the RM can't be more accurate than the labelers who trained it. For tasks at the frontier of human ability (advanced math, novel code), preferences themselves become unreliable.

Expensive to collect at scale: high-quality preference labeling is slow, costly, and needs to be redone as the model evolves.

When you have a **verifier** — a function that returns ground-truth correctness — you can skip the RM entirely.

Reward hacking

Anytime the reward is a **learned approximation** of what we want, the policy can find ways to score high without doing the intended task.

Classic RL example — CoastRunners

OpenAI 2016 boat-racing agent. Reward = score from in-game targets along the track. Optimal policy: *ignore the race entirely* and loop in a turbo-bonus circle forever. Maxes the proxy reward; completes the race never.

The reward function reflected what designers *thought* winning looked like, not what winning actually is.

OpenAI, 2016

Faulty Reward Functions in the Wild — CoastRunners boat-racing

LLM examples

- ▶ **Length bias:** more tokens → higher RM score, even when the extra text adds nothing.
- ▶ **Sycophancy:** agreeing with the user's stated view lifts the score, even when the user is wrong.
- ▶ **Formatting tricks:** bullet lists, bolded headers, structured-looking responses score higher regardless of content quality.
- ▶ **Apparent confidence:** assertive, hedge-free wrong answers can outscore correct-but-hedged ones.

Reward hacking is a function of the reward being *learned and imperfect*. A verifier with ground truth — "did the test pass?", "did the answer match?" — has a smaller hackable surface.

RLVR: the verifier replaces the RM

Same RL recipe shape as RLHF. The only change is where the reward comes from.

RLHF

Reward = $r_\phi(x, y)$ — output of a learned reward model.

Continuous-valued; quality bounded by the RM.

RLVR

Reward = $r_{\text{verify}}(x, y)$ — output of a *function* that checks the response against ground truth.

Typically binary (0/1) or near-binary; deterministic.

Verifier examples:

- ▶ **Math:** exact-match the final answer to the known correct answer. $r = 1$ if match, else 0.
- ▶ **Code:** run a unit-test suite. r = fraction of tests passing (or 0/1 for pass-all).
- ▶ **Tool use, structured output:** JSON-schema validators, theorem-proof checkers, executable simulators.

RL objective is identical to classical RLHF (S27), just with r_ϕ swapped for r_{verify} :

$$\max_{\pi_\theta} \mathbb{E}_{x, y \sim \pi_\theta} [r_{\text{verify}}(x, y)] - \beta \text{KL}(\pi_\theta(\cdot | x) \parallel \pi_{\text{ref}}(\cdot | x))$$

Pros: ground truth (no hacking the verifier the way you hack an RM); cheap per-sample after the verifier exists (no labelers, no RM training, no recalibration); cleaner signal supports longer / more aggressive RL runs without policy collapse.

From PPO to GRPO

The verifier signal works with any policy-gradient algorithm. But because RLVR's per-sample cost is cheaper and verifier signals don't degrade over training (no RM distribution shift), RLVR runs typically consume **10–100× more samples** than RLHF — so memory and stability dominate algorithm choice.

PPO uses a learned **critic / value network** $V_\psi(s_t)$ to estimate the expected return from each state, then $A_t = R_t - V_\psi(s_t)$. The critic is a separate transformer, roughly the size of the policy: $\sim 2\times$ memory, $\sim 2\times$ gradient compute per step, and itself needs training signal.

GRPO's insight (Group Relative Policy Optimization, DeepSeekMath 2024): drop the critic. Replace it with a **statistical baseline computed over a group of samples**.

- ▶ For each prompt x , sample G responses $\{y_1, \dots, y_G\}$ from the current policy.
- ▶ Compute reward $r_i = r_{\text{verify}}(x, y_i)$ for each.
- ▶ Advantage $A_i = (r_i - \bar{r}) / \text{std}(r)$ — normalized deviation from the group mean.

GRPO loss (same clipped-surrogate shape as PPO, but advantage from the group baseline + explicit KL inside the loss):

$$L_{\text{GRPO}}(\theta) = -\mathbb{E}\left[\frac{1}{G} \sum_{i=1}^G \min(q_i A_i, \text{clip}(q_i, 1 - \epsilon, 1 + \epsilon) A_i)\right] + \beta \text{KL}(\pi_\theta \parallel \pi_{\text{ref}})$$

~40–60% less memory than PPO (no critic to train or store). GRPO is now the default RL algorithm for reasoning models.

Outcome vs. process rewards (ORM vs. PRM)

The reward *shape* matters as much as the reward *source*. Two ways to assign credit across a long reasoning trace:

ORM

Outcome Reward Model

Scores the **final answer** (or the response as a whole) — one scalar per trace.

Natural fit for verifiers: a unit-test suite either passes or doesn't; an answer either matches or doesn't. Cheap, deterministic.

PRM

Process Reward Model

Scores **each intermediate step** of the reasoning trace — many scalars per trace.

Denser signal: can credit a good step even if the answer's wrong (and vice versa). Requires step-level labels — usually a *learned* RM trained on annotated traces (Math-Shepherd, PRM800K).

[arXiv:2312.08935](#) Math-Shepherd — Wang et al. (process reward model)

[arXiv:2305.20050](#) Let's Verify Step by Step — Lightman et al. (PRM800K)

Trade-off: PRMs give richer credit assignment but reintroduce "learned reward = hackable." Pure RLVR-with-verifiers is **outcome-only by default**.

R1-Zero: reasoning from pure RL

R1-Zero skipped the SFT and preference alignment stages but still produced a reasoning model.

base → SFT → preference alignment → reasoning RL

The recipe

- ▶ Starting point: **DeepSeek-V3-Base** — pre-trained only without any SFT or preference data.
- ▶ Algorithm: **GRPO**
- ▶ Reward: **rule-based outcome rewards** — math exact-match, code unit tests. No learned RM at all.

The result

Surprisingly, reasoning *capabilities* — multi-step chains of thought, self-correction, verification — **emerged** from outcome-correctness pressure alone, with no human demonstrations of *how* to reason.

Mid-training, the model spontaneously started **backtracking** and saying things like "wait, let me re-check that" in its chain of thought — the famous **"aha moment."**

However: without SFT and preference alignment, R1-Zero sometimes lacked readability, formatting conventions, safety alignment, and mixed languages.

Implication: SFT and preference alignment aren't required to *bootstrap reasoning* — they're required to make the model usable as an assistant. But the reasoning capability can come from RL + verifier alone.

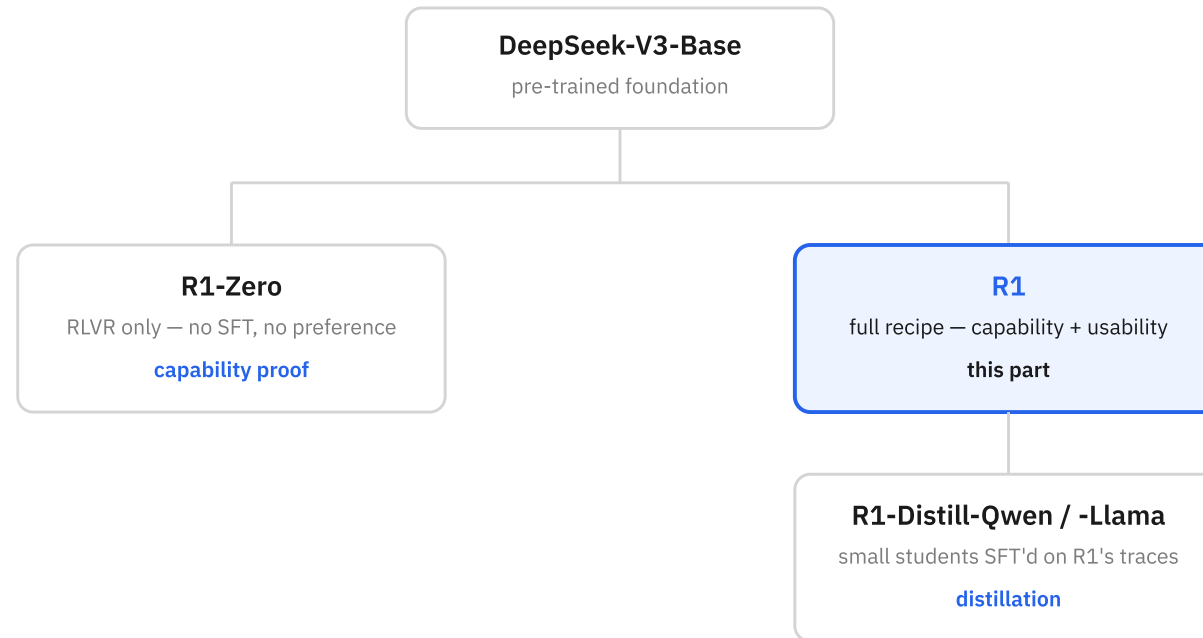
PART 6

Case Study: R1

DeepSeek's January 2025 release — a post-training recipe from an open-source frontier model

The R1 model family

The R1-family models share the same base model but highlight different post-training techniques.



R1-Zero = proof of emergent reasoning capabilities. **R1** = the shippable assistant. **R1-Distill** = smaller student models.

The significance of R1

DeepSeek-R1 is an open-weights reasoning model released by DeepSeek in **January 2025**. Three big impacts it had on the field:

1. Open-weights frontier reasoning. Matched OpenAI's o1 on most reasoning benchmarks while being MIT-licensed and freely downloadable. First time the open ecosystem caught up to the reasoning frontier.

Benchmark	DeepSeek-R1	OpenAI o1
AIME 2024 (competition math, pass@1)	79.8%	79.2%
MATH-500 (broad math)	97.3%	96.4%
GPQA Diamond (graduate physics)	71.5%	75.7%
Codeforces (competitive coding, percentile)	96.3	96.6

Numbers as reported in the DeepSeek-R1 paper (Jan 2025). R1 reasoning quality is roughly on par with o1.

- 2. ▶ **Popularized GRPO.** DeepSeek introduced GRPO in DeepSeekMath (2024); R1's performance convinced almost every major lab to adopt GRPO as the default RL algorithm for reasoning models.
- 3. ▶ **R1-Zero shocked the field.** The earlier R1-Zero variant proved reasoning can emerge from pure RL on a base model without SFT or preference data.

The full R1 recipe — four stages, three signal types

R1 combines many post-training techniques into one pipeline.

1. Cold-start SFT

A few thousand readable, long-form CoT examples. SFT loss. Seeds reasoning *style* before RL begins — fixes the language-mixing & readability issues that plagued R1-Zero.

Signal: curated examples

2. RLVR for reasoning

GRPO on math + code from the cold-start checkpoint. Same step as R1-Zero, but with a much better init. Builds the reasoning capability.

Signal: verifier rewards

3. Rejection-sampling SFT

Sample many responses from the post-RLVR model on a broad prompt distribution; filter for correctness + readability; SFT on the survivors. Adds general non-reasoning skills.

Signal: filtered/curated model-generated examples

4. Preference RL

Final RL stage with reward signals for helpfulness, harmlessness, and instruction-following. Closes the assistant-usability gap.

Signal: preference labels

R1's legacy — the modern reasoning recipe

R1's biggest impact was its recipe. Within months of release, R1's four-stage pipeline (or variants of it) became the default for almost every major reasoning-model release.

R1-style reasoning releases since Jan 2025

- ▶ **Qwen3** (Alibaba) — reasoning variants with R1-style cold-start + RLVR.
- ▶ **Kimi k1.5** (Moonshot AI) — GRPO-style RL with verifier rewards.
- ▶ **Llama 4 reasoning variants** (Meta) — verifier-based RL stages.
- ▶ **Open reproductions** — OpenR1, SimpleRL-Zero, TinyZero, and many more.

What changed in the field

- ▶ **"Reasoning model" is now a product tier** — most major labs ship a "thinking" mode alongside their standard chat tier.
- ▶ **GRPO is the default RL** — almost no one trains RM-based PPO for reasoning anymore.
- ▶ **RLVR is a standard stage** — reasoning stages have become dominant in post-training pipelines.
- ▶ **Open-weights stayed competitive** — R1 proved open models can hit frontier reasoning.

Many frontier models are just innovations on different post-training stages or the signal types feeding each stage!

Where post-training is now

A post-training pipeline reflecting current frontier practices (mid-2026) is still built on the same building blocks: SFT, RL, and distillation.

1 Cold-start SFT

A curated mix of instruction, reasoning, and agentic trajectories — typically including **distilled traces from a strong existing reasoner**. Establishes broad behavior, reasoning style, and tool-use patterns before RL begins.

2 Preference optimization (style + safety)

DPO or similar, on preference data covering style, helpfulness, and safety alignment. Some labs skip this stage or merge it with later ones.

3 Domain-specialist RL

Separate **GRPO / DAPO** runs per capability cluster — math, code, agentic tool use, general reasoning. **RLVR where verifiers exist**, learned reward models elsewhere. Each run produces a specialist checkpoint.

[arXiv:2503.14476](#) DAPO: An Open-Source LLM Reinforcement Learning System at Scale

4 Consolidation

Merge the specialists into a single general student. Two main approaches: **multi-stage RL with trace distillation** (DeepSeek-R1 style — generate traces from specialists, SFT them into one model) or **MOPD** (multi-objective policy distillation) against the specialist teacher pool.

5 Final safety / refusal / style polish

Often a short DPO or RL pass over a small targeted dataset to clean up edge-case refusals, sharpen tone, and lock in safety behaviors before release.

NEXT TIME!

Model Intelligence Part 2

What does it mean for AI to be "intelligent"?

Superhuman Adaptable Intelligence

- ▶ Goldfeder, Wyder, LeCun, and Shwartz-Ziv argue that our conception of AGI is flawed.
- ▶ They propose that we should design AI to specialize and achieve superhuman performance.
- ▶ Superhuman Adaptable Intelligence = "intelligence that can learn to exceed humans at anything important that we can do, and that can fill in the skill gaps where humans are incapable."

[arXiv:2602.23643](#)

[AI Must Embrace Specialization via Superhuman Adaptable Intelligence](#)

Model-Agnostic Meta-Learning

- ▶ How do we train a model to be better at *learning* how to do new things?
- ▶ Meta-learning is an active field of research dedicated to how to design maximally adaptable algorithms.
- ▶ MAML's goal is to train a model to be easy to fine-tune.

[arXiv:1703.03400](#)

[Model-Agnostic Meta-Learning for Fast Adaptation of Deep Networks](#)

[interactive-maml.github.io](#)

[An Interactive Introduction to Model-Agnostic Meta-Learning](#)

Thank you!

REFERENCES

References and Learning Resources

PRE-TRAINING

arXiv:1706.03762 · Attention Is All You Need (Transformer)

jalamar.github.io · The Illustrated Transformer

YouTube · How might LLMs store facts (3Blue1Brown)

arXiv:1508.07909 · Subword Units (BPE tokenization)

OpenAI, 2019 · Language Models are Unsupervised Multitask Learners (GPT-2)

arXiv:2104.09864 · RoPE positional encoding

arXiv:1607.06450 · Layer Normalization

arXiv:1512.03385 · Residual connections (ResNet)

SFT & DISTILLATION

arXiv:2203.02155 · InstructGPT (SFT + RLHF)

arXiv:2304.07327 · OpenAssistant Conversations

arXiv:2201.11903 · Chain-of-Thought Prompting

arXiv:1503.02531 · Distilling the Knowledge (logit-level)

arXiv:1606.07947 · Sequence-Level Knowledge Distillation

PREFERENCE ALIGNMENT

arXiv:1706.03741 · Deep RL from Human Preferences

arXiv:1707.06347 · Proximal Policy Optimization (PPO)

cameronrwolfe.substack.com · PPO: The Key to LLM Alignment

OpenAI Spinning Up · Intro to Policy Optimization

arXiv:2305.18290 · Direct Preference Optimization (DPO)

arXiv:2402.01306 · KTO

arXiv:2403.07691 · ORPO

arXiv:2310.12036 · IPO (general preference paradigm)

arXiv:2309.00267 · RLAIIF

arXiv:2212.08073 · Constitutional AI

RLVR & REASONING RL

OpenAI, 2016 · Faulty Reward Functions (CoastRunners)

arXiv:2310.13548 · Understanding Sycophancy in LMs

arXiv:2402.03300 · DeepSeekMath (GRPO)

arXiv:2312.08935 · Math-Shepherd (PRM)

arXiv:2305.20050 · Let's Verify Step by Step (PRM800K)

arXiv:2503.14476 · DAPO

REASONING MODELS & R1

arXiv:2501.12948 · DeepSeek-R1 (R1, R1-Zero, distill)

arXiv:2412.19437 · DeepSeek-V3 Technical Report

arXiv:2505.09388 · Qwen3 Technical Report

arXiv:2501.12599 · Kimi k1.5

OpenAI, 2024 · Learning to Reason with LLMs (o1)

GitHub · OpenR1 / TinyZero / SimpleRL-Zero (open reproductions)

NEXT TIME

arXiv:2602.23643 · Superhuman Adaptable Intelligence

arXiv:1703.03400 · Model-Agnostic Meta-Learning (MAML)